

Evolution Towards Transformers

1. Basic Encoder–Decoder Architecture

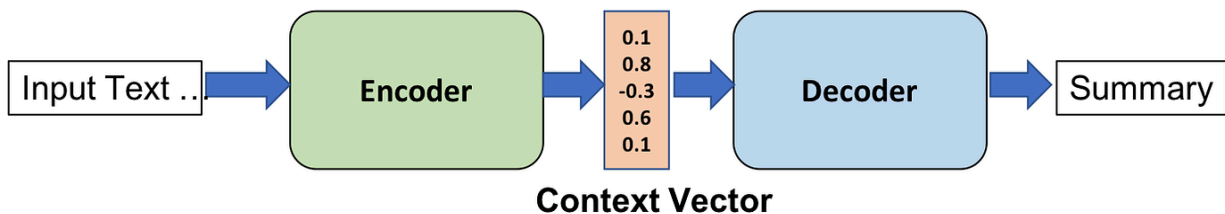
Early sequence models (e.g., RNN-based) used a simple **Encoder–Decoder framework**.

Idea

- **Encoder** → Reads input sequence and compresses it into a **fixed-size context vector**
- **Decoder** → Uses that vector to generate output sequence

Flow

1. Input sentence → converted into vectors (embeddings)
2. Encoder processes sequence → produces hidden states
3. Final hidden state → **context vector**
4. Decoder uses this vector → generates output word-by-word



2. Adding Word Embeddings (Semantic Understanding)

Instead of raw tokens, models started using embeddings from Word2Vec neural embedding model

Why?

- Captures **semantic meaning**
- Similar words → similar vectors

Improvement

- Better understanding of language
- Improved translation quality

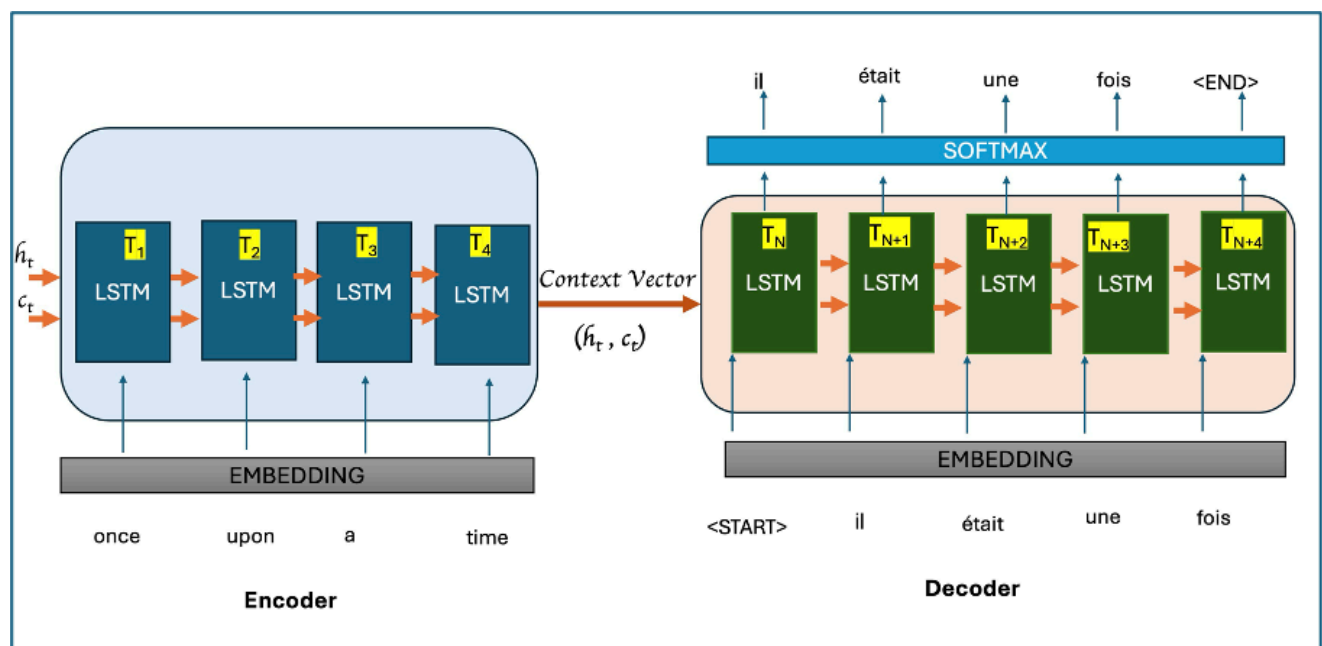
Problem with Basic Encoder–Decoder

Bottleneck: Single Context Vector

- Entire sentence compressed into **one fixed vector**
- For long sentences:
 - Information loss
 - Poor translation quality

👉 Example:

Long paragraph → squeezed into one vector → decoder struggles.



3. Attention Mechanism (Breakthrough)

Introduced in the paper:

Neural Machine Translation by Jointly Learning to Align and Translate (Bahdanau et al., 2014)

Core Idea of Attention

Instead of one context vector → use **multiple context vectors dynamically**

👉 Decoder **focuses on relevant words** at each step

How Attention Works

At each decoding step t :

1. Compare:
 - Encoder hidden states
 - Previous decoder state
2. Compute **attention weights (α)**
3. Create context vector:

$$\text{Context} = \sum \alpha_i \cdot h_i$$

Where:

- h_i = encoder hidden states
- α_i = importance of each word

Intuition

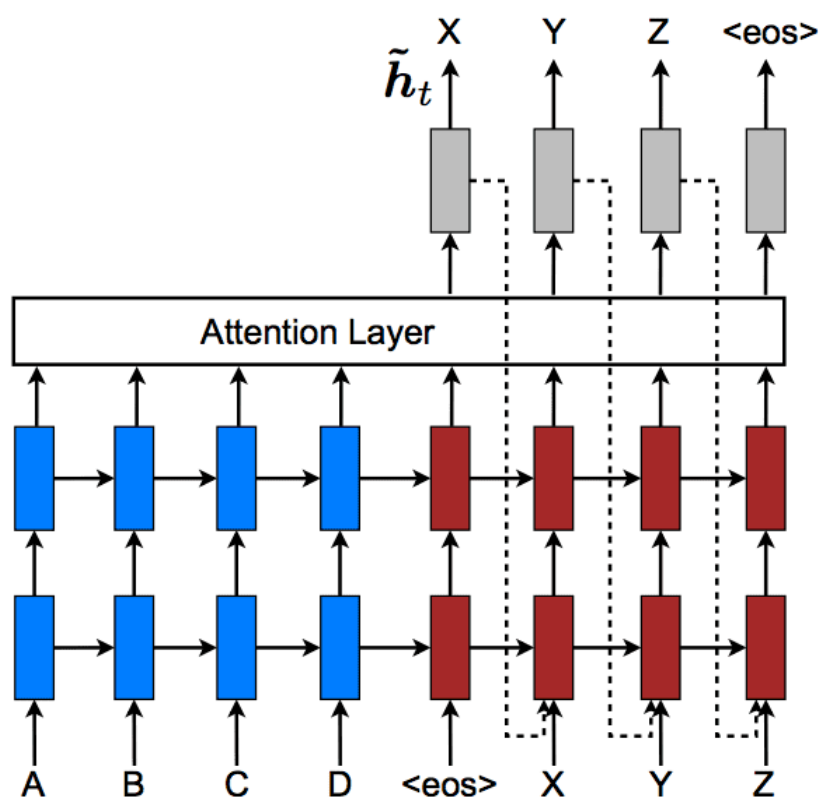
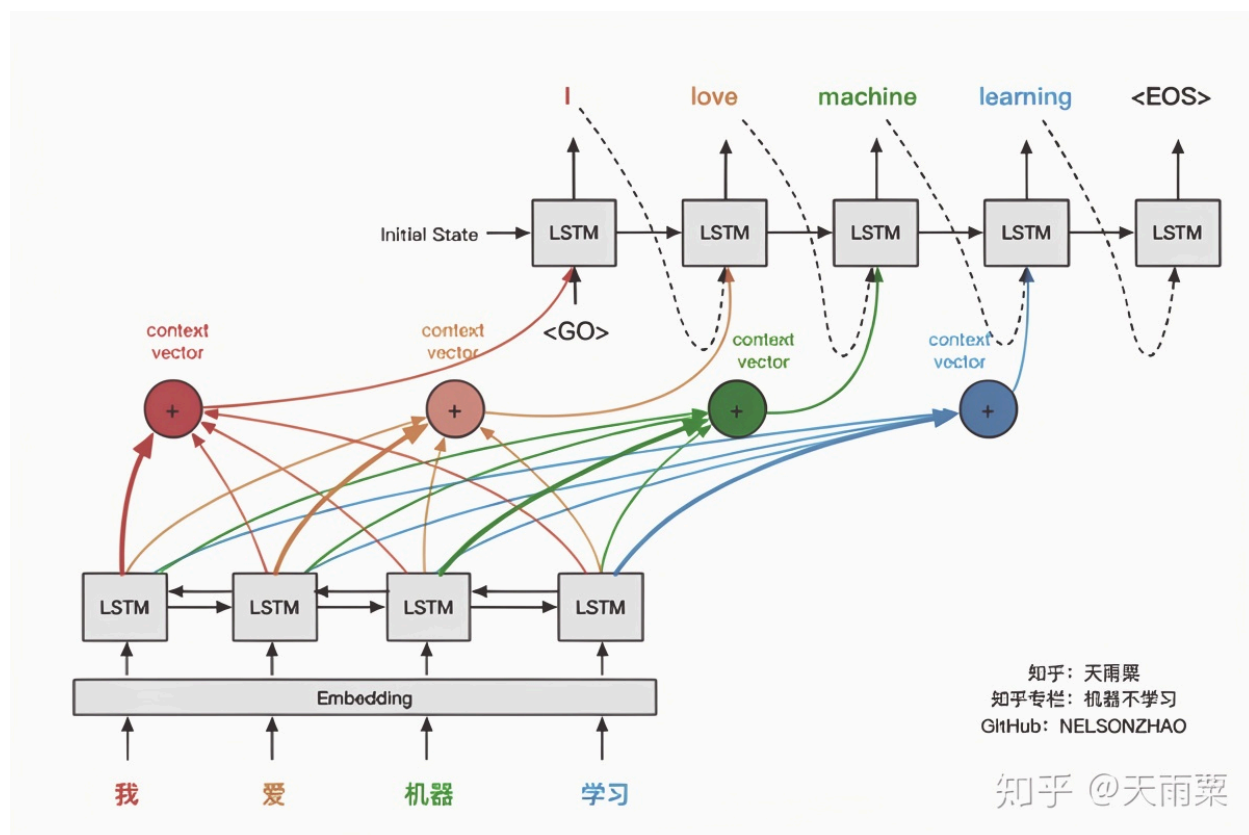
When translating:

- Not all words matter equally
- Model **“attends” to relevant words only**

Example:

Translating “The cat sat on the mat”

→ For word “cat”, focus mainly on “cat”

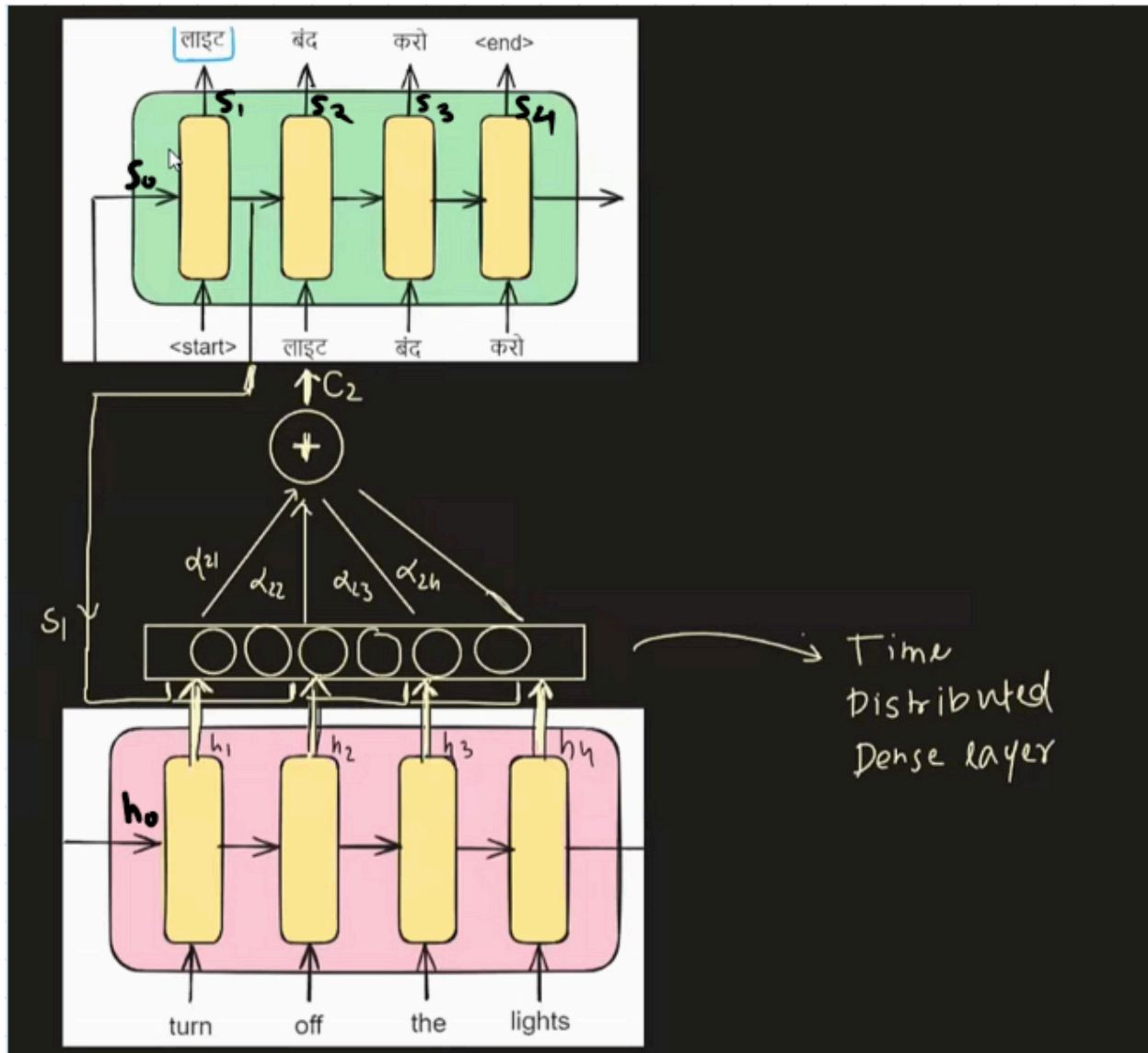


4. Types of Attention

1. Bahdanau Attention (Additive)

- Uses a small neural network
- Combines:
 - Encoder state
 - Decoder state

More flexible, but slower.

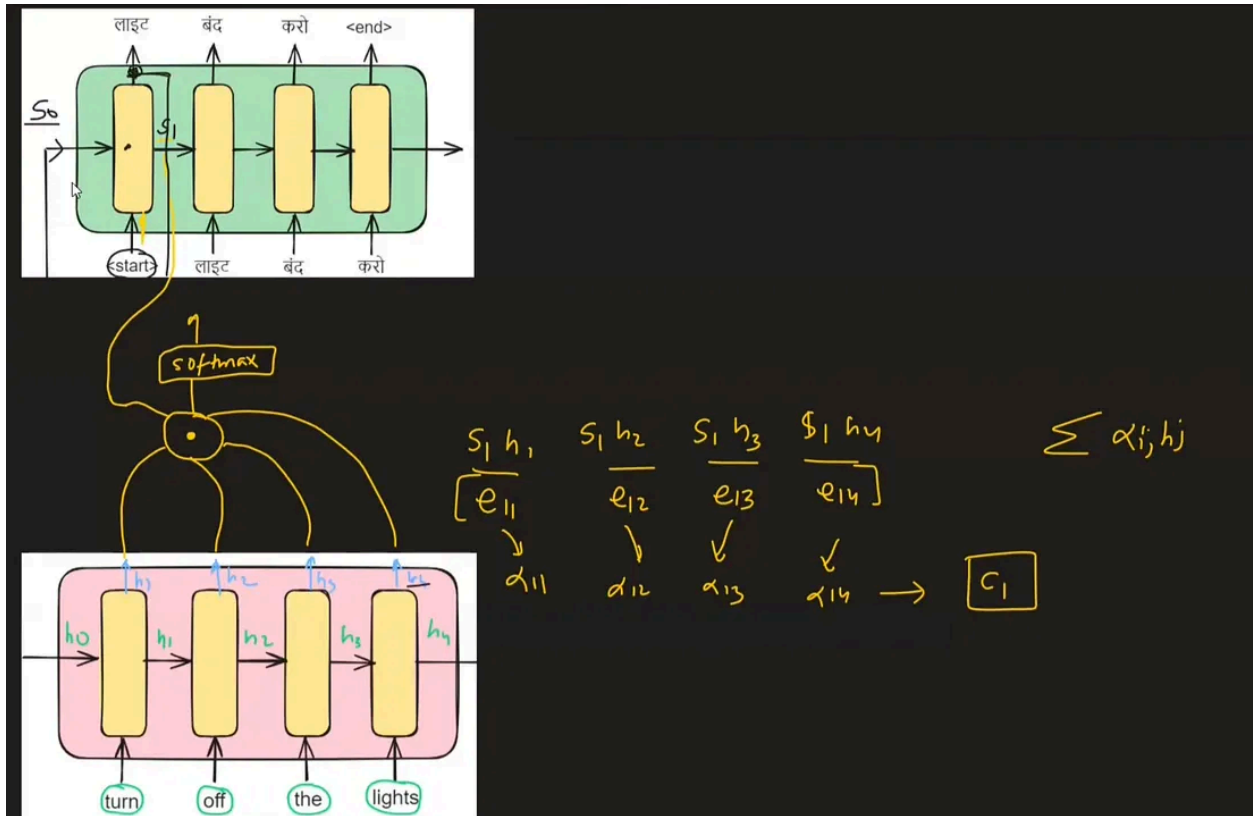


2. Luong Attention (Multiplicative / Dot Product)

- Uses **dot product** between vectors

$$\text{score} = \mathbf{h_t^T} \cdot \mathbf{s_t} - 1$$

👉 Faster and computationally efficient



5. Improvements Achieved

Solved context bottleneck
 Better handling of long sentences
 Improved translation accuracy

Remaining Problem

Sequential Nature (Major Limitation)

Even with attention:

- Processing is still **time-step based**
- Cannot parallelize
- Slow training

Why Sequential is Bad?

- RNNs process:
 $t=1 \rightarrow t=2 \rightarrow t=3$

👉 Cannot compute all steps at once

Issues Caused

1. Slow Training

- Cannot use full GPU parallel power

2. Difficult Transfer Learning

- Hard to pre-train large models
- Fine-tuning inefficient

3. Long Dependencies Still Costly

- Even with attention, computation grows

Motivation for Transformers

Researchers needed a model that:

Removes sequential dependency

Enables **parallel processing**

Handles long-range dependencies efficiently

Supports **transfer learning + fine-tuning**

6. Transition to Transformers

This led to the landmark paper:

👉 Attention Is All You Need (Vaswani et al., 2017)

Key Idea

“Remove recurrence completely and rely only on attention”

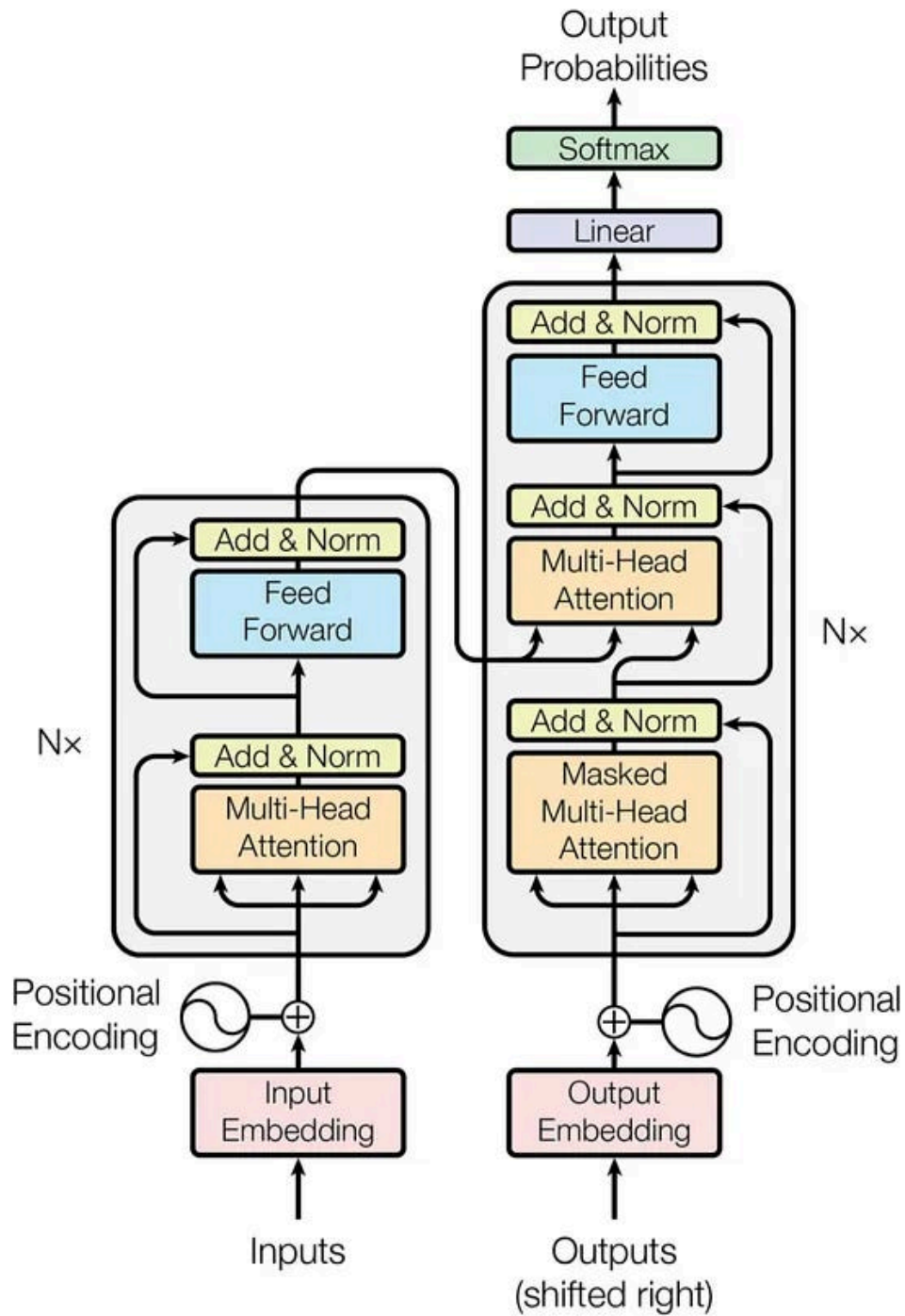


Figure 1: The Transformer - model architecture.

Why Transformers Solved the Problem

Problem	Solution in Transformers
---------	--------------------------

Sequential processing	Fully parallel computation
Single context vector	Self-attention across all words
Slow training	GPU-efficient
Weak transfer learning	Pretraining-friendly

Final Flow (Your Notes Summary)

1. Encoder–Decoder (RNN)
2. Add embeddings (Word2Vec)
3. Problem: single context vector
4. Introduce attention (Bahdanau, Luong)
5. Problem: still sequential
6. Solution: Transformers (parallel attention)